

# Minimax Algorithm

## 1. Aim

To implement the **Minimax algorithm** for decision-making in a two-player game by evaluating possible game states and selecting the optimal move for the maximizing player.

## 2. Problem Statement

In two-player games, each player tries to make the best possible move while considering the opponent's actions. The Minimax problem is to determine the **best possible move** by exploring different future game states, assuming that both players make optimal decisions.

## 3. Solution

The problem is solved using the **Minimax algorithm**, a recursive decision-making technique used in Artificial Intelligence for game playing. The algorithm considers two players: a **MAX player**, who tries to maximize the score, and a **MIN player**, who tries to minimize the score. By evaluating the possible outcomes of each move, the algorithm selects the move that provides the best result for the MAX player.

## 4. Algorithm

1. Start from the current state of the game.
2. Generate all possible moves from the current state.

3. If the state is a terminal state, return its utility value.
4. For the MAX player, select the maximum value among the possible moves.
5. For the MIN player, select the minimum value among the possible moves.
6. Recursively evaluate the child states.
7. Continue until the terminal states are reached.
8. Propagate the evaluated values back to the root node.
9. Select the move with the best value for the MAX player.
10. Display the optimal value or move.

## **5. Program**

```
def minimax(tree, maximizing_player):
```

```
    # Leaf node
```

```
    if isinstance(tree, int):
```

```
        return tree
```

```
    if maximizing_player:
```

```
        best = float('-inf')
```

```
for child in tree:
    value = minimax(child, False)
    best = max(best, value)

return best
```

```
else:
    best = float('inf')
```

```
for child in tree:
    value = minimax(child, True)
    best = min(best, value)

return best
```

```
# Game tree
```

```
tree = [
    [3, 5],
    [2, 9],
```

```
[0, 7]  
]  
  
# MAX player starts  
result = minimax(tree, True)  
  
print("Optimal Value:", result)
```

## 6. Result

The Minimax algorithm was successfully implemented using Python. The algorithm evaluated the possible outcomes of the given game tree by alternating between the **MAX and MIN players** and selected the best possible value for the MAX player.

### Output:

Optimal Value: 7

Thus, the **Minimax algorithm successfully determined the optimal value by considering the possible decisions of both players.**

